```
import express from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import dotenv from 'dotenv';
import { GoogleGenAI, Type, ThinkingLevel } from '@google/genai';

dotenv.config();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// In-memory cache for YouTube video queries (24 hours TTL)
interface VideoCacheItem {
  data: any[];
  timestamp: number;
}
const videoCache = new Map<string, VideoCacheItem>();
const CACHE_TTL_MS = 24 * 60 * 60 * 1000;

// In-memory cache for Mind Maps (24 hours TTL, normalized key)
interface MapCacheItem {
  data: any;
  timestamp: number;
}
const mapCache = new Map<string, MapCacheItem>();

function normalizeThemeKey(text: string): string {
  return text
    .toLowerCase()
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .trim();
}

// Curated high-yield fallbacks for common ENEM queries when API key is missing or quota is
exhausted
const FALLBACK_VIDEOS: Record<string, any[]> = {
  default: [
    {
      youtubeId: 'F9X6CeqsV-o',
      title: 'Como Gabaritar no ENEM: Método de Estudo e Resolução',
      channel: 'Descomplica',
      description: 'Estratégia completa para o ENEM: resolução rápida, coerência TRI e leitura.',
      thumbnail: 'https://img.youtube.com/vi/F9X6CeqsV-o/hqdefault.jpg'
    },
    {
      youtubeId: 'Gg4Zc-y9X6M',
      title: 'Revisão Geral ENEM: O que mais cai na prova',
      channel: 'Brasil Escola',
      description: 'Principais tópicos cobrados pelo INEP comentados por professores.',
      thumbnail: 'https://img.youtube.com/vi/Gg4Zc-y9X6M/hqdefault.jpg'
    }
  ],
  'meio ambiente': [
```

```
    {
      youtubeId: '8fLq7SOsA9Y',
      title: 'Ecologia e Meio Ambiente no ENEM: Resumo Completo',
      channel: 'Biologia Total com Jubilut',
      description: 'Desmatamento, mudanças climáticas, poluição e legislação ambiental explicados.',
      thumbnail: 'https://img.youtube.com/vi/8fLq7SOsA9Y/hqdefault.jpg'
    }
  ],
  'redacao': [
    {
      youtubeId: 'W4rD3GqZ9vE',
      title: 'Redação ENEM Nota 1000: Estrutura e Proposta de Intervenção',
      channel: 'Débora Aladim',
      description: 'Passo a passo dos 4 parágrafos e dos 5 elementos da Competência 5.',
      thumbnail: 'https://img.youtube.com/vi/W4rD3GqZ9vE/hqdefault.jpg'
    }
  ]
};

// ========================================
// ERROR CLASSIFIER
// ========================================
interface ClassifiedError {
  status: number;
  type: string;
  message: string;
}

export function is404Error(err: any): boolean {
  const status = Number(err?.status || err?.statusCode || 0);
  const errMsg = (err?.message || String(err)).toLowerCase();
  return (
    status === 404 ||
    errMsg.includes('404') ||
    errMsg.includes('not_found') ||
    errMsg.includes('not found') ||
    errMsg.includes('is not found') ||
    errMsg.includes('is no longer') ||
    errMsg.includes('unsupported model')
  );
}

function classifyGeminiError(err: any, context: string = 'a operação'): ClassifiedError {
  const errMsg = err?.message || String(err);
  const status = Number(err?.status || err?.statusCode || 0);

  // 1. Missing or invalid API key
  if (
    status === 401 ||
    status === 403 ||
    errMsg.includes('API_KEY_INVALID') ||
    errMsg.includes('apiKey is not valid') ||
    errMsg.includes('API key not valid') ||
    errMsg.includes('PERMISSION_DENIED') ||
    errMsg.includes('Forbidden')
  ) {
    return {
      status: 401,
```

```
      type: 'auth_error',
      message: 'Chave de API do Gemini ausente ou inválida. Configure a GEMINI_API_KEY no painel de
Secrets.'
    };
  }

  // 2. Model not found (404)
  if (is404Error(err)) {
    return {
      status: 404,
      type: 'model_not_found',
      message: 'Nenhum modelo do Gemini disponível para esta chave. Abra /api/gemini-status.'
    };
  }

  // 3. Quota exhausted (429)
  if (
    status === 429 ||
    errMsg.includes('429') ||
    errMsg.includes('RESOURCE_EXHAUSTED') ||
    errMsg.includes('quota') ||
    errMsg.includes('rate limit')
  ) {
    return {
      status: 429,
      type: 'quota_exhausted',
      message: 'Cota de requisições esgotada na API do Gemini (429). Tente novamente mais tarde.'
    };
  }

  // 4. Service overloaded (503)
  if (
    status === 503 ||
    status === 500 ||
    errMsg.includes('503') ||
    errMsg.includes('500') ||
    errMsg.includes('UNAVAILABLE') ||
    errMsg.includes('overloaded') ||
    errMsg.includes('high demand') ||
    errMsg.includes('temporarily unavailable')
  ) {
    return {
      status: 503,
      type: 'service_overloaded',
      message: 'Serviço do Gemini temporariamente sobrecarregado (503). Tente novamente em
instantes.'
    };
  }

  // 5. Timeout
  if (errMsg.includes('TIMEOUT') || errMsg.includes('tempo esgotado') || errMsg.includes('timed
out')) {
    return {
      status: 504,
      type: 'timeout',
      message: `Tempo limite esgotado ao ${context} (timeout de 50s). Tente novamente.`
    };
  }
```

```
  // 6. Invalid response / parse error
  if (errMsg.includes('INVALID_SCHEMA') || errMsg.includes('PARSE_ERROR') ||
errMsg.includes('TREE_BUILD_FAILED')) {
    return {
      status: 502,
      type: 'invalid_response',
      message: 'A IA retornou uma resposta em formato inválido. Tente novamente.'
    };
  }

  return {
    status: 500,
    type: 'server_error',
    message: `Erro na comunicação com a API do Gemini: ${errMsg.slice(0, 150)}`
  };
}

// ========================================
// TREE ASSEMBLY (FLAT NODES -> TREE)
// ========================================
interface FlatNode {
  id: string;
  parentId?: string | null;
  label: string;
  summary: string;
  enemTips?: string;
  videoQuery?: string;
}

interface TreeNode {
  id: string;
  label: string;
  summary: string;
  enemTips?: string;
  videoQuery?: string;
  children: TreeNode[];
}

function buildMindMapTree(flatNodes: FlatNode[], defaultTheme: string): TreeNode | null {
  if (!Array.isArray(flatNodes) || flatNodes.length === 0) return null;

  // Find root node index
  let rootIndex = flatNodes.findIndex(
    (n) => n && (n.id === 'root' || !n.parentId || n.parentId.trim() === '' || n.parentId === 'null'
|| n.parentId === n.id)
  );
  if (rootIndex === -1) rootIndex = 0;

  const rawRoot = flatNodes[rootIndex];
  const rootId = (rawRoot?.id && String(rawRoot.id).trim()) || 'root';

  const nodeMap = new Map<string, TreeNode>();
  const childrenMap = new Map<string, TreeNode[]>();

  for (const raw of flatNodes) {
    if (!raw || !raw.id || !raw.label) continue;
    const cleanId = String(raw.id).trim();
```

```
  if (!cleanId) continue;

  nodeMap.set(cleanId, {
    id: cleanId,
    label: String(raw.label).trim(),
    summary: String(raw.summary || raw.label).trim(),
    ...(raw.enemTips ? { enemTips: String(raw.enemTips).trim() } : {}),
    ...(raw.videoQuery ? { videoQuery: String(raw.videoQuery).trim() } : {}),
    children: [],
  });
}

for (const raw of flatNodes) {
  if (!raw || !raw.id) continue;
  const cleanId = String(raw.id).trim();
  if (cleanId === rootId) continue;

  const rawPid = raw.parentId ? String(raw.parentId).trim() : '';
  const parentId = (rawPid && rawPid !== 'null' && nodeMap.has(rawPid)) ? rawPid : rootId;

  if (!childrenMap.has(parentId)) {
    childrenMap.set(parentId, []);
  }
  const item = nodeMap.get(cleanId);
  if (item) {
    childrenMap.get(parentId)!.push(item);
  }
}

const rootNode = nodeMap.get(rootId) || {
  id: 'root',
  label: defaultTheme,
  summary: 'Visão geral e contexto no ENEM.',
  children: [],
};

// Limits: Max 4 main branches, max 30 nodes in total, max depth 4 levels
let nodeCount = 1;
const MAX_NODES = 30;
const MAX_MAIN_BRANCHES = 4;
const MAX_DEPTH = 4;

function attach(parent: TreeNode, depth: number) {
  if (depth >= MAX_DEPTH || nodeCount >= MAX_NODES) {
    parent.children = [];
    return;
  }

  const availableChildren = childrenMap.get(parent.id) || [];
  const limit = depth === 1 ? MAX_MAIN_BRANCHES : availableChildren.length;
  const slice = availableChildren.slice(0, limit);

  parent.children = [];
  for (const child of slice) {
    if (nodeCount >= MAX_NODES) break;
    nodeCount++;
    parent.children.push(child);
    attach(child, depth + 1);
```

```
    }
  }

  attach(rootNode, 1);

  // Fallback: If root ended up with 0 children, attach any available nodes directly
  if (rootNode.children.length === 0 && nodeMap.size > 1) {
    for (const [id, node] of nodeMap.entries()) {
      if (id !== rootId && rootNode.children.length < MAX_MAIN_BRANCHES) {
        rootNode.children.push(node);
      }
    }
  }

  return rootNode;
}

// ========================================
// MODELS AVAILABILITY AND RESOLUTION
// ========================================
const EXCLUDED_MODEL_SUBSTRINGS = [
  'image',
  'tts',
  'live',
  'audio',
  'embedding',
  'imagen',
  'veo',
  'robotics',
  'computer-use',
  'aqa'
];

// In-memory 404 cache: models that returned 404 are excluded for 10 minutes
const failed404Models = new Map<string, number>();
const NOT_FOUND_CACHE_TTL_MS = 10 * 60 * 1000; // 10 minutes

function markModelNotFound(modelName: string): void {
  const clean = modelName.replace(/^models\//, '').trim();
  failed404Models.set(clean, Date.now());
  console.warn(`[Gemini 404 Cache] Modelo "${clean}" retornou 404. Removido dos candidatos por 10
minutos.`);
}

function isModel404Excluded(modelName: string): boolean {
  const clean = modelName.replace(/^models\//, '').trim();
  const failedAt = failed404Models.get(clean);
  if (!failedAt) return false;
  if (Date.now() - failedAt < NOT_FOUND_CACHE_TTL_MS) {
    return true;
  }
  failed404Models.delete(clean);
  return false;
}

interface ModelsDiscoveryResult {
  models: string[];
  succeeded: boolean;
```

```
  timestamp: number;
}
let availableModelsCache: ModelsDiscoveryResult | null = null;
const MODELS_CACHE_TTL = 5 * 60 * 1000;

async function fetchAvailableModels(ai: GoogleGenAI, forceFresh = false): Promise<{ models:
string[]; succeeded: boolean }> {
  const now = Date.now();
  if (!forceFresh && availableModelsCache && (now - availableModelsCache.timestamp <
MODELS_CACHE_TTL)) {
    return { models: availableModelsCache.models, succeeded: availableModelsCache.succeeded };
  }

  try {
    const pager = await ai.models.list();
    const modelsList: string[] = [];
    for await (const m of pager) {
      if (!m?.name) continue;
      const cleanName = m.name.replace(/^models\//, '').trim();
      const lower = cleanName.toLowerCase();

      // Exclude models with forbidden keywords
      const isExcluded = EXCLUDED_MODEL_SUBSTRINGS.some((sub) => lower.includes(sub));
      if (isExcluded) continue;

      // Only consider models that support generateContent
      const actions: string[] = (m as any).supportedActions || (m as any).supportedGenerationMethods
|| [];
      if (actions.length > 0 && !actions.includes('generateContent')) {
        continue;
      }

      modelsList.push(cleanName);
    }

    availableModelsCache = {
      models: modelsList,
      succeeded: true,
      timestamp: now
    };
    return { models: modelsList, succeeded: true };
  } catch (err: any) {
    console.warn('[Gemini Warning] Falha ao listar modelos via ai.models.list():', err?.message ||
err);
    return { models: [], succeeded: false };
  }
}

function extractModelVersion(name: string): number {
  const match = name.match(/gemini-(\d+(\.\d+)?)/i);
  if (match && match[1]) {
    return parseFloat(match[1]);
  }
  const anyNum = name.match(/(\d+(\.\d+)?)/);
  return anyNum ? parseFloat(anyNum[0]) : 0;
}

function resolveConfiguredModels(
```

```
  discoveryInput: { models: string[]; succeeded: boolean } | string[],
  options?: { forVision?: boolean }
): {
  primaryModel: string;
  fallbackModels: string[];
  activeModel: string;
  orderedCandidates: string[];
  isPrimaryAvailable: boolean;
  totalDiscovered: number;
  discoverySucceeded: boolean;
} {
  const models = Array.isArray(discoveryInput)
    ? discoveryInput
    : (discoveryInput?.models || []);
  const discoverySucceeded = Array.isArray(discoveryInput)
    ? true
    : Boolean(discoveryInput?.succeeded);

  const primaryModel = (process.env.GEMINI_MODEL || 'gemini-flash-latest').trim();
  const fallbackEnv = process.env.GEMINI_FALLBACK_MODELS || 'gemini-flash-lite-latest, gemini-2.5-
flash, gemini-2.5-flash-lite';
  const fallbackModels = fallbackEnv
    .split(',')
    .map((m) => m.trim())
    .filter(Boolean);

  let candidatePool: string[] = [];

  if (discoverySucceeded && models.length > 0) {
    // REQUIREMENT 3: "NUNCA tente um modelo que não esteja na lista retornada pela API."
    candidatePool = models.filter((m) => !isModel404Excluded(m));
  } else {
    // REQUIREMENT 3: "Se models.list() falhar, use apenas os apelidos 'gemini-flash-latest' e
'gemini-flash-lite-latest' e as variáveis de ambiente."
    candidatePool = Array.from(new Set([
      primaryModel,
      'gemini-flash-latest',
      'gemini-flash-lite-latest',
      ...fallbackModels
    ])).filter((m) => !isModel404Excluded(m));
  }

  // Vision requirement: "Para a transcrição de fotos, use a mesma ordem, mas só modelos que aceitem
imagem."
  if (options?.forVision) {
    candidatePool = candidatePool.filter((m) => m.toLowerCase().startsWith('gemini-'));
  }

  // REQUIREMENT 2: Ordem de preferência para texto (JSON):
  // 1) primeiro o que estiver em GEMINI_MODEL, se EXISTIR na lista;
  // 2) depois "gemini-flash-latest";
  // 3) depois os modelos "flash" estáveis (sem "preview"/"exp") do mais novo para o mais antigo;
  // 4) depois os "flash-lite";
  // 5) por último os "preview" flash.
  // 6) quaisquer outros modelos descobertos remanescentes.

  const orderedCandidates: string[] = [];
  const added = new Set<string>();
```

```
  const addCandidate = (m: string) => {
    if (m && candidatePool.includes(m) && !added.has(m)) {
      orderedCandidates.push(m);
      added.add(m);
    }
  };

  // 1) primeiro o que estiver em GEMINI_MODEL, se EXISTIR na lista
  if (primaryModel && candidatePool.includes(primaryModel)) {
    addCandidate(primaryModel);
  }

  // 2) depois "gemini-flash-latest"
  if (candidatePool.includes('gemini-flash-latest')) {
    addCandidate('gemini-flash-latest');
  }

  // 3) depois os modelos "flash" estáveis (sem "preview"/"exp" e sem "lite") do mais novo para o
mais antigo
  const flashStable = candidatePool
    .filter((m) => {
      const l = m.toLowerCase();
      return l.includes('flash') && !l.includes('preview') && !l.includes('exp') &&
!l.includes('lite') && m !== 'gemini-flash-latest';
    })
    .sort((a, b) => extractModelVersion(b) - extractModelVersion(a));
  flashStable.forEach(addCandidate);

  // 4) depois os "flash-lite"
  if (candidatePool.includes('gemini-flash-lite-latest')) {
    addCandidate('gemini-flash-lite-latest');
  }
  const flashLite = candidatePool
    .filter((m) => {
      const l = m.toLowerCase();
      return l.includes('flash') && l.includes('lite') && !l.includes('preview') &&
!l.includes('exp') && m !== 'gemini-flash-lite-latest';
    })
    .sort((a, b) => extractModelVersion(b) - extractModelVersion(a));
  flashLite.forEach(addCandidate);

  // 5) por último os "preview" flash
  const flashPreview = candidatePool
    .filter((m) => {
      const l = m.toLowerCase();
      return l.includes('flash') && (l.includes('preview') || l.includes('exp'));
    })
    .sort((a, b) => extractModelVersion(b) - extractModelVersion(a));
  flashPreview.forEach(addCandidate);

  // 6) quaisquer outros modelos descobertos remanescentes
  const remanescentes = candidatePool
    .filter((m) => !added.has(m))
    .sort((a, b) => extractModelVersion(b) - extractModelVersion(a));
  remanescentes.forEach(addCandidate);

  const activeModel = orderedCandidates[0] || '';
```

```
  const isPrimaryAvailable = Boolean(primaryModel && candidatePool.includes(primaryModel));

  return {
    primaryModel,
    fallbackModels,
    activeModel,
    orderedCandidates,
    isPrimaryAvailable,
    totalDiscovered: models.length,
    discoverySucceeded
  };
}

async function createServer() {
  const app = express();
  const PORT = process.env.PORT || 3000;

  app.use(express.json({ limit: '20mb' }));
  app.use(express.urlencoded({ extended: true, limit: '20mb' }));

  // 1) GET /api/videos?q=...
  app.get('/api/videos', async (req, res) => {
    const rawQuery = (req.query.q as string) || '';
    if (!rawQuery.trim()) {
      return res.status(400).json({ error: 'Parâmetro de busca "q" é obrigatório.' });
    }

    const searchTerm = rawQuery.toLowerCase().includes('enem')
      ? rawQuery.trim()
      : `${rawQuery.trim()} ENEM`;

    const cacheKey = searchTerm.toLowerCase();
    const now = Date.now();

    // Check memory cache
    if (videoCache.has(cacheKey)) {
      const cached = videoCache.get(cacheKey)!;
      if (now - cached.timestamp < CACHE_TTL_MS) {
        return res.json({ query: searchTerm, source: 'cache', videos: cached.data });
      }
    }

    const apiKey = process.env.YOUTUBE_API_KEY;

    // If API key is not present, use graceful search fallback
    if (!apiKey || apiKey === 'MY_YOUTUBE_API_KEY') {
      const keyWord = Object.keys(FALLBACK_VIDEOS).find((k) => cacheKey.includes(k)) || 'default';
      const fallbackList = FALLBACK_VIDEOS[keyWord] || FALLBACK_VIDEOS.default;
      return res.json({
        query: searchTerm,
        source: 'fallback',
        videos: fallbackList,
        message: 'Chave YOUTUBE_API_KEY não configurada. Exibindo videoaula recomendada.',
        searchUrl: `https://www.youtube.com/results?search_query=${encodeURIComponent(searchTerm)}`
      });
    }

    try {
```

```
      const url = new URL('https://www.googleapis.com/youtube/v3/search');
      url.searchParams.set('part', 'snippet');
      url.searchParams.set('type', 'video');
      url.searchParams.set('videoEmbeddable', 'true');
      url.searchParams.set('relevanceLanguage', 'pt');
      url.searchParams.set('q', searchTerm);
      url.searchParams.set('maxResults', '5');
      url.searchParams.set('key', apiKey);

      const ytResponse = await fetch(url.toString());
      if (!ytResponse.ok) {
        const errorText = await ytResponse.text();
        console.warn('YouTube API error response:', ytResponse.status, errorText);
        return res.json({
          query: searchTerm,
          source: 'fallback',
          videos: FALLBACK_VIDEOS.default,
          searchUrl: `https://www.youtube.com/results?
search_query=${encodeURIComponent(searchTerm)}`
        });
      }

      const ytData = await ytResponse.json();
      const items = ytData.items || [];

      const parsedVideos = items
        .filter((item: any) => item.id?.videoId)
        .map((item: any) => ({
          youtubeId: item.id.videoId,
          title: item.snippet.title,
          channel: item.snippet.channelTitle,
          description: item.snippet.description,
          thumbnail: item.snippet.thumbnails?.high?.url || item.snippet.thumbnails?.default?.url
        }));

      videoCache.set(cacheKey, { data: parsedVideos, timestamp: now });
      return res.json({ query: searchTerm, source: 'youtube-api', videos: parsedVideos });
    } catch (err: any) {
      console.error('Error fetching YouTube videos:', err);
      return res.json({
        query: searchTerm,
        source: 'fallback',
        videos: FALLBACK_VIDEOS.default,
        searchUrl: `https://www.youtube.com/results?search_query=${encodeURIComponent(searchTerm)}`
      });
    }
  });

  // 2) GET /api/gemini-status
  app.get('/api/gemini-status', async (_req, res) => {
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey || apiKey === 'MY_GEMINI_API_KEY') {
      return res.status(401).json({
        status: 'error',
        error: 'Chave de API do Gemini ausente ou inválida. Configure a GEMINI_API_KEY no painel de
Secrets.'
      });
    }
```

```
try {
  const ai = new GoogleGenAI({
    apiKey,
    httpOptions: {
      headers: {
        'User-Agent': 'aistudio-build',
      },
    },
  });

  const discovery = await fetchAvailableModels(ai, true);
  const resolution = resolveConfiguredModels(discovery);

  if (resolution.orderedCandidates.length === 0) {
    return res.status(404).json({
      status: 'error',
      activeModel: null,
      orderedCandidates: [],
      error: 'Nenhum modelo do Gemini disponível para esta chave. Abra /api/gemini-status.',
      excluded404Models: Array.from(failed404Models.keys()).filter((m) => isModel404Excluded(m))
    });
  }

  const activeModel = resolution.activeModel;

  // Test call with active model: "responda ok"
  let testCallResult: {
    tested: boolean;
    success: boolean;
    modelUsed: string;
    response?: string;
    error?: string;
  };

  try {
    const testResp = await ai.models.generateContent({
      model: activeModel,
      contents: 'responda ok',
      config: {
        maxOutputTokens: 20,
        temperature: 0.1,
      },
    });
    testCallResult = {
      tested: true,
      success: true,
      modelUsed: activeModel,
      response: (testResp.text || '').trim(),
    };
  } catch (testErr: any) {
    if (is404Error(testErr)) {
      markModelNotFound(activeModel);
    }
    testCallResult = {
      tested: true,
      success: false,
      modelUsed: activeModel,
```

```
          error: testErr?.message || String(testErr)
        };
      }

      return res.json({
        status: testCallResult.success ? 'ok' : 'degraded',
        activeModel,
        orderedCandidates: resolution.orderedCandidates,
        testCall: testCallResult,
        configuredPrimary: resolution.primaryModel,
        configuredFallbacks: resolution.fallbackModels,
        isPrimaryAvailable: resolution.isPrimaryAvailable,
        totalDiscoveredModels: resolution.totalDiscovered,
        excluded404Models: Array.from(failed404Models.keys()).filter((m) => isModel404Excluded(m)),
        message: testCallResult.success
          ? `Modelo ativo "${activeModel}" testado e funcionando com sucesso.`
          : `Aviso: teste com o modelo ativo "${activeModel}" falhou (${testCallResult.error}).`
      });
    } catch (err: any) {
      const classified = classifyGeminiError(err, 'verificar o status do modelo');
      console.error(`[Gemini Status Real Error] Status: ${err?.status || 500}, Detail:
${err?.message || err}`);
      return res.status(classified.status).json({
        status: 'error',
        error: classified.message,
        detail: err?.message || String(err)
      });
    }
  });

  // 3) POST /api/mapa { tema }
  app.post('/api/mapa', async (req, res) => {
    const REQUEST_START = Date.now();
    const TOTAL_TIMEOUT_MS = 50000; // 50 seconds total deadline

    const { tema } = req.body;
    if (!tema || typeof tema !== 'string' || !tema.trim()) {
      return res.status(400).json({ error: 'O campo "tema" é obrigatório.' });
    }

    const cleanTema = tema.trim();
    const cacheKey = normalizeThemeKey(cleanTema);
    const now = Date.now();

    // Check 24-hour cache
    if (mapCache.has(cacheKey)) {
      const cached = mapCache.get(cacheKey)!;
      if (now - cached.timestamp < CACHE_TTL_MS) {
        return res.json({
          tema: cleanTema,
          mapa: cached.data,
          source: 'cache'
        });
      }
    }

    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey || apiKey === 'MY_GEMINI_API_KEY') {
```

```
      console.error('[Gemini Real Error] GEMINI_API_KEY ausente ou não configurada no ambiente.');
      return res.status(401).json({
        error: 'Chave de API do Gemini ausente ou inválida. Configure a GEMINI_API_KEY no painel de
Secrets.'
      });
    }

    const ai = new GoogleGenAI({
      apiKey,
      httpOptions: {
        headers: {
          'User-Agent': 'aistudio-build',
        },
      },
    });

    // Check available models from API
    const discovery = await fetchAvailableModels(ai, false);
    const { orderedCandidates } = resolveConfiguredModels(discovery);

    if (orderedCandidates.length === 0) {
      return res.status(404).json({
        error: 'Nenhum modelo do Gemini disponível para esta chave. Abra /api/gemini-status.',
        type: 'model_not_found'
      });
    }

    const prompt = `Você é o especialista pedagógico da plataforma "ENEM Nota Max".
Crie um mapa mental plano sobre o tema: "${cleanTema}".

ESTRUTURA OBRIGATÓRIA (LISTA PLANA DE NÓS):
1. O primeiro nó DEVE ser a raiz: { "id": "root", "parentId": null, "label": "${cleanTema}",
"summary": "Visão geral e contexto pedagógico no ENEM." }.
2. Crie EXATAMENTE de 3 a 4 ramos principais conectados à raiz com "parentId": "root".
3. Cada ramo principal deve se desdobrar em subtemas e detalhes, totalizando até 4 níveis de
profundidade (raiz > ramo principal > subtema > detalhe).
4. No MÁXIMO 30 nós no total em toda a lista.
5. Cada nó deve conter:
   - "id": identificador único e conciso (ex: "root", "ramo_1", "ramo_1_1").
   - "parentId": ID do nó pai ao qual ele se conecta (use null ou "" apenas para o nó "root").
   - "label": título do nó direto e conciso (máximo 4 a 6 palavras).
   - "summary": explicação pedagógica clara e direta em 1 a 3 frases.
   - "enemTips": dica prática de como o assunto cai no ENEM, pegadinha ou padrão de questão (coloque
nos nós mais importantes).
   - "videoQuery": termo de busca no YouTube para videoaula real (coloque em no máximo 1 a 2 nós
mais importantes por ramo principal).
6. Conteúdo 100% focado no ENEM, com explicações corretas, sem dados inventados. Quando não houver
número exato, use tendências ("mais da metade").`;

    const flatResponseSchema = {
      type: Type.ARRAY,
      description: "Lista plana de nós do mapa mental com no máximo 30 nós e até 4 níveis de
profundidade.",
      items: {
        type: Type.OBJECT,
        properties: {
          id: { type: Type.STRING, description: "ID único do nó (ex: 'root', 'ramo_1')" },
          parentId: { type: Type.STRING, description: "ID do nó pai, ou null/vazio para a raiz" },
```

```
          label: { type: Type.STRING, description: "Título do nó (máximo 4 a 6 palavras)" },
          summary: { type: Type.STRING, description: "Explicação pedagógica clara e direta em 1 a 3
frases" },
          enemTips: { type: Type.STRING, description: "Como cai no ENEM, pegadinha ou padrão de
questão (opcional)" },
          videoQuery: { type: Type.STRING, description: "Termo de busca para videoaula real no
YouTube (opcional)" }
        },
        required: ["id", "label", "summary"]
      }
    };

    let lastError: any = null;
    let lastModelAttempted = orderedCandidates[0] || 'gemini-3.7-flash';

    for (const modelName of orderedCandidates) {
      lastModelAttempted = modelName;
      let attempt = 0;
      const maxAttempts = 2; // 2 tentativas por modelo

      while (attempt < maxAttempts) {
        attempt++;

        // Check if total 50s deadline has expired
        const elapsed = Date.now() - REQUEST_START;
        if (elapsed >= TOTAL_TIMEOUT_MS) {
          console.error(`[Gemini Real Error] Model: "${modelName}", Status: 504, Message: "Tempo
total de 50s esgotado."`);
          return res.status(504).json({
            error: 'Tempo limite esgotado ao gerar o mapa mental (timeout de 50s). Tente novamente.'
          });
        }

        // Per-attempt timeout capped by remaining overall time (max 18s, so a slow
        // model can't burn the whole 50s budget and starve the fallback candidates)
        const remainingTime = TOTAL_TIMEOUT_MS - elapsed;
        const attemptTimeoutMs = Math.min(remainingTime, 18000);

        try {
          const timeoutPromise = new Promise<never>((_, reject) => {
            const timer = setTimeout(() => {
              reject(new Error('TIMEOUT_ATTEMPT'));
            }, attemptTimeoutMs);
            if (typeof timer.unref === 'function') timer.unref();
          });

          // Low thinking level for speed and broad model compatibility
          const thinkingLevel = ThinkingLevel.LOW;

          const generatePromise = ai.models.generateContent({
            model: modelName,
            contents: prompt,
            config: {
              responseMimeType: 'application/json',
              thinkingConfig: { thinkingLevel },
              maxOutputTokens: 2500,
              responseSchema: flatResponseSchema,
              temperature: 0.2,
```

```
            },
          });

          const response = await Promise.race([generatePromise, timeoutPromise]);
          const rawText = response.text || '';

          let flatNodes: FlatNode[];
          try {
            flatNodes = JSON.parse(rawText);
          } catch (jsonErr) {
            console.warn(`[Gemini Real Error] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: PARSE_ERROR, Message: "Falha ao analisar JSON retornado."`);
            lastError = new Error('PARSE_ERROR');
            if (attempt < maxAttempts) {
              await new Promise((r) => setTimeout(r, 1000 + Math.random() * 1000));
              continue;
            }
            break;
          }

          if (!Array.isArray(flatNodes) || flatNodes.length === 0) {
            console.warn(`[Gemini Real Error] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: INVALID_SCHEMA, Message: "Resposta não é um array de nós."`);
            lastError = new Error('INVALID_SCHEMA');
            if (attempt < maxAttempts) {
              await new Promise((r) => setTimeout(r, 1000 + Math.random() * 1000));
              continue;
            }
            break;
          }

          // Build hierarchical tree from flat nodes
          const tree = buildMindMapTree(flatNodes, cleanTema);
          if (!tree || !tree.label || !Array.isArray(tree.children) || tree.children.length === 0) {
            console.warn(`[Gemini Real Error] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: TREE_BUILD_FAILED, Message: "Árvore hierárquica vazia após
montagem."`);
            lastError = new Error('TREE_BUILD_FAILED');
            if (attempt < maxAttempts) {
              await new Promise((r) => setTimeout(r, 1000 + Math.random() * 1000));
              continue;
            }
            break;
          }

          // Success: cache valid result for 24h
          mapCache.set(cacheKey, { data: tree, timestamp: now });

          return res.json({
            tema: cleanTema,
            mapa: tree,
            modelUsed: modelName,
            source: 'gemini-api'
          });
        } catch (err: any) {
          lastError = err;
          const status = err?.status || err?.statusCode || 0;
          const errMsg = err?.message || String(err);
```

```
          // Log real status and message to server console with model name
          console.error(`[Gemini Real Error] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: ${status}, Message: "${errMsg}"`);

          const classified = classifyGeminiError(err, 'gerar o mapa mental');

          // If auth error (missing/invalid key), stop immediately and return 401
          if (classified.status === 401) {
            return res.status(401).json({ error: classified.message });
          }

          // If model not found (404), mark in cache for 10 minutes and skip to next candidate
immediately
          if (is404Error(err)) {
            markModelNotFound(modelName);
            break;
          }

          // A per-attempt timeout means this model is currently slow/congested: retrying the
          // SAME model would just burn the shared 50s budget again. Move to the next candidate.
          if (errMsg.includes('TIMEOUT_ATTEMPT')) {
            break;
          }

          // If retryable (503, 429, 500, UNAVAILABLE), wait 1 to 2 seconds before attempt 2
          const isRetryable =
            status === 503 ||
            status === 429 ||
            status === 500 ||
            errMsg.includes('503') ||
            errMsg.includes('429') ||
            errMsg.includes('500') ||
            errMsg.includes('UNAVAILABLE');

          if (!isRetryable) {
            // Other client errors: move to next model
            break;
          }

          if (attempt < maxAttempts) {
            const waitMs = 1000 + Math.floor(Math.random() * 1000); // 1 a 2 segundos
            await new Promise((r) => setTimeout(r, waitMs));
          }
        }
      }
    }

    // If all models failed or timed out, return specific classified error
    const finalClassification = classifyGeminiError(lastError, 'gerar o mapa mental');
    console.error(`[Gemini Final Failure] Theme: "${cleanTema}", Last Model:
"${lastModelAttempted}", Classification: ${finalClassification.type}
(${finalClassification.status})`);

    return res.status(finalClassification.status).json({
      error: finalClassification.message,
      type: finalClassification.type,
      modelUsed: lastModelAttempted
```

```
    });
  });

  // ========================================
  // HELPER: Base64 byte size calculation
  // ========================================
  function getBase64ByteSize(base64String: string): number {
    const clean = base64String.replace(/^data:image\/[a-zA-Z+]+base64,/, '');
    return Math.ceil((clean.length * 3) / 4);
  }

  function roundToMultipleOf40(val: number): number {
    const clamped = Math.max(0, Math.min(200, val));
    const remainder = clamped % 40;
    if (remainder >= 20) {
      return Math.min(200, clamped + (40 - remainder));
    } else {
      return Math.max(0, clamped - remainder);
    }
  }

  // 4) POST /api/redacao/transcrever
  app.post('/api/redacao/transcrever', async (req, res) => {
    const REQUEST_START = Date.now();
    const TOTAL_TIMEOUT_MS = 50000;

    const { images } = req.body;
    if (!images || !Array.isArray(images) || images.length === 0) {
      return res.status(400).json({ error: 'Nenhuma imagem foi enviada para transcrição.' });
    }

    if (images.length > 3) {
      return res.status(400).json({ error: 'Envie no máximo 3 imagens (páginas) da folha de
redação.' });
    }

    // Validate 4MB limit per photo
    const MAX_PHOTO_BYTES = 4 * 1024 * 1024; // 4 MB
    for (let i = 0; i < images.length; i++) {
      const img = images[i];
      const rawData = typeof img === 'string' ? img : img?.data;
      if (!rawData || typeof rawData !== 'string') {
        return res.status(400).json({ error: `Foto ${i + 1} inválida ou corrompida.` });
      }

      const byteSize = getBase64ByteSize(rawData);
      if (byteSize > MAX_PHOTO_BYTES) {
        const mb = (byteSize / (1024 * 1024)).toFixed(1);
        const name = typeof img === 'object' && img?.name ? `"${img.name}"` : `Foto ${i + 1}`;
        return res.status(400).json({
          error: `${name} excede o limite máximo permitido de 4 MB (tamanho detectado: ${mb} MB).
Reduza a resolução ou comprima a imagem antes de enviar.`
        });
      }
    }

    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey || apiKey === 'MY_GEMINI_API_KEY') {
```

```
      console.error('[Gemini Real Error] GEMINI_API_KEY ausente ou não configurada no ambiente.');
      return res.status(401).json({
        error: 'Chave de API do Gemini ausente ou inválida. Configure a GEMINI_API_KEY no painel de
Secrets.'
      });
    }

    const ai = new GoogleGenAI({
      apiKey,
      httpOptions: {
        headers: {
          'User-Agent': 'aistudio-build',
        },
      },
    });

    const discovery = await fetchAvailableModels(ai, false);
    const { orderedCandidates } = resolveConfiguredModels(discovery, { forVision: true });

    if (orderedCandidates.length === 0) {
      return res.status(404).json({
        error: 'Nenhum modelo do Gemini disponível para esta chave. Abra /api/gemini-status.',
        type: 'model_not_found'
      });
    }

    const promptText = `Você é um transcritor paleógrafo e pedagógico de alta precisão especializado
em caligrafia manuscrita de redações do ENEM.
Transcreva com máxima fidelidade o texto manuscrito presente nas imagens fornecidas da folha de
redação.

REGRAS OBRIGATÓRIAS:
1. Transcreva exatamente o que o aluno escreveu, letra por letra e palavra por palavra, preservando
pontuação e eventuais desvios ortográficos ou gramaticais. NÃO corrija os erros do aluno.
2. Preserve estritamente os parágrafos e as quebras de linha originais da folha de redação.
3. Se houver qualquer palavra ilegível, rasurada ou de caligrafia duvidosa, você DEVE
obrigatoriamente marcá-la entre [?...?] (por exemplo: [?sociedade?], [?adotar?], [?
desenvolvimento?]).
4. NÃO inclua saudações, cabeçalhos, comentários, explicações ou notas. Retorne EXCLUSIVAMENTE o
texto transcrito da redação.`;

    // Prepare multimodal parts
    const contentParts: any[] = [];
    for (const img of images) {
      const rawData = typeof img === 'string' ? img : img?.data;
      const cleanData = rawData.replace(/^data:image\/[a-zA-Z+]+base64,/, '');
      let mimeType = 'image/jpeg';
      if (typeof img === 'object' && img?.mimeType) {
        mimeType = img.mimeType;
      } else if (rawData.startsWith('data:image/png')) {
        mimeType = 'image/png';
      } else if (rawData.startsWith('data:image/webp')) {
        mimeType = 'image/webp';
      }

      contentParts.push({
        inlineData: {
          mimeType,
```

```
          data: cleanData
        }
      });
    }
    contentParts.push({ text: promptText });

    let lastError: any = null;
    let lastModelAttempted = orderedCandidates[0] || 'gemini-3.7-flash';

    for (const modelName of orderedCandidates) {
      lastModelAttempted = modelName;
      let attempt = 0;
      const maxAttempts = 2;

      while (attempt < maxAttempts) {
        attempt++;

        const elapsed = Date.now() - REQUEST_START;
        if (elapsed >= TOTAL_TIMEOUT_MS) {
          console.error(`[Gemini Real Error] Transcrição Model: "${modelName}", Status: 504,
Message: "Tempo total de 50s esgotado."`);
          return res.status(504).json({
            error: 'Tempo limite esgotado ao transcrever a redação manuscrita (timeout de 50s).
Tente novamente.'
          });
        }

        // Image transcription needs a bit more room than pure text, but still capped
        // well below the full 50s so fallback models get a real chance.
        const remainingTime = TOTAL_TIMEOUT_MS - elapsed;
        const attemptTimeoutMs = Math.min(remainingTime, 22000);

        try {
          const timeoutPromise = new Promise<never>((_, reject) => {
            const timer = setTimeout(() => {
              reject(new Error('TIMEOUT_ATTEMPT'));
            }, attemptTimeoutMs);
            if (typeof timer.unref === 'function') timer.unref();
          });

          const thinkingLevel = ThinkingLevel.LOW;

          const generatePromise = ai.models.generateContent({
            model: modelName,
            contents: { parts: contentParts },
            config: {
              thinkingConfig: { thinkingLevel },
              maxOutputTokens: 2500,
              temperature: 0.1,
            },
          });

          const response = await Promise.race([generatePromise, timeoutPromise]);
          const transcribedText = (response.text || '').trim();

          if (!transcribedText) {
            console.warn(`[Gemini Real Error] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: EMPTY_RESPONSE`);
```

```
            lastError = new Error('INVALID_SCHEMA');
            if (attempt < maxAttempts) {
              await new Promise((r) => setTimeout(r, 1000 + Math.random() * 1000));
              continue;
            }
            break;
          }

          return res.json({
            texto: transcribedText,
            modelUsed: modelName,
            source: 'gemini-vision'
          });
        } catch (err: any) {
          lastError = err;
          const status = err?.status || err?.statusCode || 0;
          const errMsg = err?.message || String(err);

          console.error(`[Gemini Real Error Transcrição] Model: "${modelName}", Attempt:
${attempt}/${maxAttempts}, Status: ${status}, Message: "${errMsg}"`);

          const classified = classifyGeminiError(err, 'transcrever a redação manuscrita');
          if (classified.status === 401) {
            return res.status(401).json({ error: classified.message });
          }

          if (is404Error(err)) {
            markModelNotFound(modelName);
            break;
          }

          // A per-attempt timeout means this model is currently slow/congested: retrying the
          // SAME model would just burn the shared 50s budget again. Move to the next candidate.
          if (errMsg.includes('TIMEOUT_ATTEMPT')) {
            break;
          }

          const isRetryable =
            status === 503 ||
            status === 429 ||
            status === 500 ||
            errMsg.includes('503') ||
            errMsg.includes('429') ||
            errMsg.includes('500') ||
            errMsg.includes('UNAVAILABLE');

          if (!isRetryable) {
            break;
          }

          if (attempt < maxAttempts) {
            const waitMs = 1000 + Math.floor(Math.random() * 1000);
            await new Promise((r) => setTimeout(r, waitMs));
          }
        }
      }
    }
```

```
    const finalClassification = classifyGeminiError(lastError, 'transcrever a redação manuscrita');
    return res.status(finalClassification.status).json({
      error: finalClassification.message,
      type: finalClassification.type,
      modelUsed: lastModelAttempted
    });
  });

  // 5) POST /api/redacao/analisar { tema, texto }
  app.post('/api/redacao/analisar', async (req, res) => {
    const REQUEST_START = Date.now();
    const TOTAL_TIMEOUT_MS = 50000;

    const { tema, texto } = req.body;
    if (!tema || typeof tema !== 'string' || !tema.trim()) {
      return res.status(400).json({ error: 'O tema da redação é obrigatório.' });
    }
    if (!texto || typeof texto !== 'string' || !texto.trim()) {
      return res.status(400).json({ error: 'O texto da redação é obrigatório para análise.' });
    }

    const cleanTema = tema.trim();
    const cleanTexto = texto.trim();

    // Check lines and words for immediate ENEM zero-score rule
    const lines = cleanTexto.split('\n').filter((l) => l.trim().length > 0);
    const wordCount = cleanTexto.split(/\s+/).filter(Boolean).length;

    // Rule: <= 7 lines is automatically zeroed by ENEM regulations
    if (lines.length <= 7 && wordCount < 60) {
      return res.json({
        tema: cleanTema,
        texto: cleanTexto,
        notaTotal: 0,
        anulacao: {
          aplica: true,
          motivo: 'Texto insuficiente (até 7 linhas). O edital oficial do ENEM determina nota 0
imediata para textos com 7 linhas ou menos.'
        },
        competencias: [
          { id: 'C1', nome: 'Domínio da Norma Culta', nota: 0, justificativa: 'Redação anulada por
extensão insuficiente (até 7 linhas).', trechos: [] },
          { id: 'C2', nome: 'Compreensão da Proposta e Repertório', nota: 0, justificativa: 'Redação
anulada por extensão insuficiente (até 7 linhas).', trechos: [] },
          { id: 'C3', nome: 'Seleção e Organização de Ideias', nota: 0, justificativa: 'Redação
anulada por extensão insuficiente (até 7 linhas).', trechos: [] },
          { id: 'C4', nome: 'Coesão e Mecanismos Linguísticos', nota: 0, justificativa: 'Redação
anulada por extensão insuficiente (até 7 linhas).', trechos: [] },
          { id: 'C5', nome: 'Proposta de Intervenção e Direitos Humanos', nota: 0, justificativa:
'Redação anulada por extensão insuficiente (até 7 linhas).', trechos: [] }
        ],
        comentarioGeral: 'A redação não atingiu o limite mínimo de 8 linhas estipulado pela matriz
de avaliação do ENEM e recebeu nota 0.',
        avaliadores: {
          avaliador1: { total: 0, notas: { C1: 0, C2: 0, C3: 0, C4: 0, C5: 0 } },
          avaliador2: { total: 0, notas: { C1: 0, C2: 0, C3: 0, C4: 0, C5: 0 } },
          houveDiscrepancia: false
        },
```

```
        dataIso: new Date().toISOString()
      });
    }

    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey || apiKey === 'MY_GEMINI_API_KEY') {
      console.error('[Gemini Real Error] GEMINI_API_KEY ausente ou não configurada no ambiente.');
      return res.status(401).json({
        error: 'Chave de API do Gemini ausente ou inválida. Configure a GEMINI_API_KEY no painel de
Secrets.'
      });
    }

    const ai = new GoogleGenAI({
      apiKey,
      httpOptions: {
        headers: {
          'User-Agent': 'aistudio-build',
        },
      },
    });

    const discovery = await fetchAvailableModels(ai, false);
    const { orderedCandidates } = resolveConfiguredModels(discovery);

    if (orderedCandidates.length === 0) {
      return res.status(404).json({
        error: 'Nenhum modelo do Gemini disponível para esta chave. Abra /api/gemini-status.',
        type: 'model_not_found'
      });
    }

    const redacaoSchema = {
      type: Type.OBJECT,
      properties: {
        anulacao: {
          type: Type.OBJECT,
          description: "Casos oficiais de anulação da redação do ENEM",
          properties: {
            aplica: { type: Type.BOOLEAN, description: "True se a redação deve ser zerada
integralmente" },
            motivo: { type: Type.STRING, description: "Motivo exato da anulação caso se aplique, ou
string vazia se válida" }
          },
          required: ["aplica", "motivo"]
        },
        competencias: {
          type: Type.ARRAY,
          description: "Avaliação rigorosa das 5 competências do ENEM",
          items: {
            type: Type.OBJECT,
            properties: {
              id: { type: Type.STRING, description: "C1, C2, C3, C4 ou C5" },
              nome: { type: Type.STRING, description: "Nome oficial da competência" },
              nota: { type: Type.INTEGER, description: "Nota atribuída estritamente em múltiplos de
40: 0, 40, 80, 120, 160 ou 200" },
              justificativa: { type: Type.STRING, description: "Justificativa pedagógica concisa de
1 a 3 frases baseada nos critérios do INEP" },
```

```
              trechos: {
                type: Type.ARRAY,
                description: "Trechos literais do texto do aluno identificando desvios ou
oportunidades",
                items: {
                  type: Type.OBJECT,
                  properties: {
                    trecho: { type: Type.STRING, description: "Citação literal do trecho do texto do
aluno" },
                    problema: { type: Type.STRING, description: "Explicação do erro, desvio ou
fragilidade" },
                    sugestao: { type: Type.STRING, description: "Sugestão prática de reescrita ou
correção" }
                  },
                  required: ["trecho", "problema", "sugestao"]
                }
              }
            },
            required: ["id", "nome", "nota", "justificativa", "trechos"]
          }
        },
        comentarioGeral: {
          type: Type.STRING,
          description: "Diagnóstico geral com pontos fortes, principais fragilidades e
direcionamento para a nota máxima"
        }
      },
      required: ["anulacao", "competencias", "comentarioGeral"]
    };

    const buildEvaluatorPrompt = (evaluatorNumber: number) => `Você é o Avaliador Oficial
${evaluatorNumber} da banca de redação do ENEM.
Realize uma correção rigorosa, técnica e independente do texto dissertativo-argumentativo do
participante, seguindo à risca a Matriz de Correção do ENEM e a Cartilha do Participante do INEP.

TEMA DA REDAÇÃO: "${cleanTema}"

TEXTO DA REDAÇÃO:
"""
${cleanTexto}
"""

REGRAS DE PONTUAÇÃO E CRITÉRIOS OBRIGATÓRIOS:
1. ANULAÇÃO (Nota 0 integral em toda a prova):
   Aplique nota 0 total se ocorrer:
   - Fuga total ao tema proposto;
   - Fuga ao tipo dissertativo-argumentativo (ex: poema, narrativa, carta);
   - Texto com até 7 linhas;
   - Parte deliberadamente desconectada do tema (mensagens jocosas, receitas culinárias, hinos,
recados);
   - Desrespeito flagrante e intencional aos direitos humanos;
   - Impropérios, desenhos, formas deliberadas de anulação ou sinais gráficos isolados;
   - Cópia pura dos textos motivadores sem desenvolvimento autoral.
   Caso se aplique, configure "anulacao": { "aplica": true, "motivo": "..." } e atribua nota 0 a
todas as competências.

2. AS 5 COMPETÊNCIAS (NOTAS ESTRITAMENTE MÚLTIPLOS DE 40: 0, 40, 80, 120, 160 ou 200):
   - C1 (0 a 200): Domínio da modalidade escrita formal da Língua Portuguesa (desvios gramaticais,
```

```
ortografia, pontuação, crase, concordância verbal/nominal, regência e precisão vocabular).
   - C2 (0 a 200): Compreensão da proposta, tipo dissertativo-argumentativo e uso de repertório sociocultural legitimado, pertinente e produtivo.
   - C3 (0 a 200): Seleção, relação, organização e interpretação de informações, fatos e argumentos em defesa de um ponto de vista (projeto de texto estratégico, coerência e autoria).
   - C4 (0 a 200): Demonstração de conhecimento dos mecanismos linguísticos (coesão inter e intraparágrafos, repertório de conectivos diversificado, sem repetições).
   - C5 (0 a 200): Elaboração de proposta de intervenção com respeito aos direitos humanos e avaliação detalhada dos 5 elementos obrigatórios: 1. Agente, 2. Ação, 3. Modo/Meio, 4. Efeito/Finalidade, 5. Detalhamento de um dos elementos.

3. TRECHOS COM FEEDBACK CONSTRUTIVO:
   Para cada competência, extraia os trechos literais mais relevantes onde o participante cometeu desvios ou que merecem aprimoramento, explicando o problema e sugerindo como reescrever.`;

    // Function to run a single evaluator with fallback and retries
    async function runEvaluator(evaluatorNum: number, temp: number): Promise<any> {
      let lastEvalError: any = null;
      for (const modelName of orderedCandidates) {
        let attempt = 0;
        const maxAttempts = 2;

        while (attempt < maxAttempts) {
          attempt++;
          const elapsed = Date.now() - REQUEST_START;
          if (elapsed >= TOTAL_TIMEOUT_MS) {
            throw new Error('TIMEOUT');
          }

          // Capped below 50s so one slow model attempt can't consume the whole shared
          // budget for both evaluators running in parallel.
          const remainingTime = TOTAL_TIMEOUT_MS - elapsed;
          const attemptTimeoutMs = Math.min(remainingTime, 20000);

          try {
            const timeoutPromise = new Promise<never>((_, reject) => {
              const timer = setTimeout(() => {
                reject(new Error('TIMEOUT_ATTEMPT'));
              }, attemptTimeoutMs);
              if (typeof timer.unref === 'function') timer.unref();
            });

            const thinkingLevel = ThinkingLevel.LOW;

            const generatePromise = ai.models.generateContent({
              model: modelName,
              contents: buildEvaluatorPrompt(evaluatorNum),
              config: {
                responseMimeType: 'application/json',
                thinkingConfig: { thinkingLevel },
                maxOutputTokens: 2500,
                responseSchema: redacaoSchema,
                temperature: temp,
              },
            });

            const response = await Promise.race([generatePromise, timeoutPromise]);
            const rawJson = response.text || '';
```

```
            const parsed = JSON.parse(rawJson);

            if (parsed && Array.isArray(parsed.competencias) && parsed.competencias.length === 5) {
              return { parsed, modelName };
            }
            throw new Error('INVALID_SCHEMA');
          } catch (err: any) {
            lastEvalError = err;
            const status = err?.status || err?.statusCode || 0;
            const errMsg = err?.message || String(err);
            console.error(`[Gemini Avaliador ${evaluatorNum}] Model: "${modelName}", Attempt:
${attempt}, Status: ${status}, Msg: "${errMsg}"`);

            if (status === 401) {
              throw err;
            }

            if (is404Error(err)) {
              markModelNotFound(modelName);
              break;
            }

            // A per-attempt timeout almost always means this model is currently slow/congested.
            // Retrying the SAME model just burns the shared 50s budget twice for nothing — move
            // straight to the next candidate model instead.
            if (errMsg.includes('TIMEOUT_ATTEMPT')) {
              break;
            }

            // Other retryable errors (503/429/500): brief wait, then retry same model once.
            const isRetryable =
              status === 503 ||
              status === 429 ||
              status === 500 ||
              errMsg.includes('503') ||
              errMsg.includes('429') ||
              errMsg.includes('500') ||
              errMsg.includes('UNAVAILABLE');

            if (!isRetryable) {
              break;
            }

            if (attempt < maxAttempts) {
              await new Promise((r) => setTimeout(r, 1000 + Math.random() * 1000));
            }
          }
        }
      }
      throw lastEvalError || new Error('EVALUATOR_FAILED');
    }

    try {
      // Run Evaluator 1 and Evaluator 2 in parallel
      const [res1, res2] = await Promise.all([
        runEvaluator(1, 0.15),
        runEvaluator(2, 0.25)
      ]);
```

```
      const eval1 = res1.parsed;
      const eval2 = res2.parsed;

      // Check if either declared anulação with a legitimate reason
      if (eval1.anulacao?.aplica || eval2.anulacao?.aplica) {
        const motivo = eval1.anulacao?.motivo || eval2.anulacao?.motivo || 'Fuga ao tema ou
desrespeito aos critérios fundamentais da prova.';
        return res.json({
          tema: cleanTema,
          texto: cleanTexto,
          notaTotal: 0,
          anulacao: { aplica: true, motivo },
          competencias: (eval1.competencias || []).map((c: any) => ({
            ...c,
            nota: 0,
            justificativa: `Redação anulada pela banca: ${motivo}`
          })),
          comentarioGeral: `A redação foi considerada anulada pela banca examinadora. Motivo:
${motivo}`,
          avaliadores: {
            avaliador1: { total: 0, notas: {} },
            avaliador2: { total: 0, notas: {} },
            houveDiscrepancia: false
          },
          dataIso: new Date().toISOString()
        });
      }

      // Compute competence maps and totals
      const compMap1: Record<string, number> = {};
      const compMap2: Record<string, number> = {};
      let total1 = 0;
      let total2 = 0;

      for (const c of eval1.competencias) {
        const score = roundToMultipleOf40(Number(c.nota) || 0);
        compMap1[c.id] = score;
        total1 += score;
      }
      for (const c of eval2.competencias) {
        const score = roundToMultipleOf40(Number(c.nota) || 0);
        compMap2[c.id] = score;
        total2 += score;
      }

      // Check for ENEM discrepancy:
      // total difference > 100 OR difference in any single competence > 80
      const totalDiff = Math.abs(total1 - total2);
      let maxCompDiff = 0;
      const compIds = ['C1', 'C2', 'C3', 'C4', 'C5'];
      for (const cid of compIds) {
        const diff = Math.abs((compMap1[cid] || 0) - (compMap2[cid] || 0));
        if (diff > maxCompDiff) maxCompDiff = diff;
      }

      let eval3: any = null;
      let selectedPair = [eval1, eval2];
```

```
      let houveDiscrepancia = false;

      if (totalDiff > 100 || maxCompDiff > 80) {
        houveDiscrepancia = true;
        console.log(`[ENEM Discrepância] Dif total: ${totalDiff}, Dif comp max: ${maxCompDiff}.
Convocando 3º Avaliador...`);
        try {
          const res3 = await runEvaluator(3, 0.20);
          eval3 = res3.parsed;

          const compMap3: Record<string, number> = {};
          let total3 = 0;
          for (const c of eval3.competencias) {
            const score = roundToMultipleOf40(Number(c.nota) || 0);
            compMap3[c.id] = score;
            total3 += score;
          }

          // Compare the pairs: (1,2), (1,3), (2,3) to find the two closest scores
          const d12 = Math.abs(total1 - total2);
          const d13 = Math.abs(total1 - total3);
          const d23 = Math.abs(total2 - total3);

          if (d13 <= d12 && d13 <= d23) {
            selectedPair = [eval1, eval3];
          } else if (d23 <= d12 && d23 <= d13) {
            selectedPair = [eval2, eval3];
          } else {
            selectedPair = [eval1, eval2];
          }
        } catch (e3: any) {
          console.warn('[Gemini Warning] Falha no 3º avaliador, mantendo média dos 2 primeiros:',
e3?.message);
        }
      }

      // Calculate final scores by competence: average of the two selected evaluators, rounded to
nearest multiple of 40
      const finalCompetencias: any[] = [];
      let finalNotaTotal = 0;

      const compNames: Record<string, string> = {
        C1: 'Domínio da Norma Culta',
        C2: 'Compreensão da Proposta e Repertório',
        C3: 'Seleção e Organização de Ideias',
        C4: 'Coesão e Mecanismos Linguísticos',
        C5: 'Proposta de Intervenção e Direitos Humanos'
      };

      for (const cid of compIds) {
        const itemA = selectedPair[0].competencias.find((c: any) => c.id === cid) || { nota: 120,
justificativa: '', trechos: [] };
        const itemB = selectedPair[1].competencias.find((c: any) => c.id === cid) || { nota: 120,
justificativa: '', trechos: [] };

        const notaA = roundToMultipleOf40(Number(itemA.nota) || 0);
        const notaB = roundToMultipleOf40(Number(itemB.nota) || 0);
        const avg = (notaA + notaB) / 2;
```

```
        const finalNota = roundToMultipleOf40(avg);

        finalNotaTotal += finalNota;

        // Combine trechos and deduplicate
        const rawTrechos = [...(itemA.trechos || []), ...(itemB.trechos || [])];
        const uniqueTrechos: any[] = [];
        const seenTrechos = new Set<string>();

        for (const tr of rawTrechos) {
          if (!tr || !tr.trecho) continue;
          const key = tr.trecho.trim().toLowerCase();
          if (!seenTrechos.has(key)) {
            seenTrechos.add(key);
            uniqueTrechos.push({
              trecho: String(tr.trecho).trim(),
              problema: String(tr.problema || '').trim(),
              sugestao: String(tr.sugestao || '').trim()
            });
          }
        }

        const justificativa = itemA.justificativa || itemB.justificativa || 'Avaliação fundamentada
na matriz oficial do ENEM.';

        finalCompetencias.push({
          id: cid,
          nome: compNames[cid] || `Competência ${cid}`,
          nota: finalNota,
          justificativa,
          trechos: uniqueTrechos.slice(0, 5) // Top 5 relevant excerpts
        });
      }

      // Consolidate general comment
      const comentarioGeral = eval1.comentarioGeral || eval2.comentarioGeral || 'Redação estruturada
com clareza e argumentação consistente.';

      return res.json({
        tema: cleanTema,
        texto: cleanTexto,
        notaTotal: finalNotaTotal,
        anulacao: { aplica: false, motivo: '' },
        competencias: finalCompetencias,
        comentarioGeral,
        avaliadores: {
          avaliador1: { total: total1, notas: compMap1 },
          avaliador2: { total: total2, notas: compMap2 },
          ...(eval3 ? { avaliador3: { total: eval3.competencias.reduce((acc: number, c: any) => acc
+ (Number(c.nota) || 0), 0) } } : {}),
          houveDiscrepancia
        },
        modelUsed: res1.modelName || 'gemini-flash',
        dataIso: new Date().toISOString()
      });
    } catch (err: any) {
      console.error('[Gemini Redação Análise Error]', err);
      const classified = classifyGeminiError(err, 'analisar a redação');
```

```
      return res.status(classified.status).json({
        error: classified.message,
        type: classified.type
      });
    }
  });

  // Serve static assets or vite middlewares
  if (process.env.NODE_ENV === 'production') {
    app.use(express.static(path.resolve(__dirname, 'dist')));
    app.get('*', (req, res) => {
      res.sendFile(path.resolve(__dirname, 'dist', 'index.html'));
    });
  } else {
    const { createServer: createViteServer } = await import('vite');
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: 'spa',
    });
    app.use(vite.middlewares);
  }

  app.listen(PORT, () => {
    console.log(`ENEM Nota Max server running on port ${PORT}`);
  });
}

createServer().catch((err) => {
  console.error('Server failed to start:', err);
});
```